IMPLEMENTATION FIELD GUIDE

GPU-Enabled Colab-to-VS Code Phase-Wise Implementation Plan

Production-oriented delivery blueprint for the ML Engineer Challenge

Execution pattern	Google Colab GPU → artifact handoff → VS Code / Docker
Primary serving target	FastAPI with ONNX Runtime CPU; TensorRT on a target GPU later
Version	1.0 | 18 July 2026

Audience: ML engineering, backend, platform, QA, and reviewers

Executive decision

THE SHORT ANSWER

Use Google Colab only for interactive GPU-dependent development—training, CUDA validation, mixed-precision runs, GPU export checks, calibration experiments, and benchmark evidence. Import the standard .ipynb notebooks into VS Code, but keep production logic in reusable Python modules. Serve the selected bundle from FastAPI, not from Colab and not directly from MLflow. On a GPU-free PC, the default serving runtime is ONNX Runtime CPU/INT8.

Non-negotiable production boundaries

- Colab is an interactive development executor, not an unattended production scheduler or public model-serving host. Managed runtimes are temporary and resource availability varies. [S2]
- A Colab-generated TensorRT engine is demonstration evidence only unless the deployment GPU, platform, CUDA and TensorRT compatibility are controlled. Build and validate the final serialized engine on the actual target GPU. [S8]
- Notebook cells orchestrate module functions; they do not contain the only copy of training, export, evaluation or packaging logic.
- MLflow records runs, artifacts and model versions and provides champion/challenger aliases. FastAPI owns online serving; promotion triggers a deployment/reload action. [S7]
- Git stores code, configuration, manifests and notebooks. Large datasets, checkpoints and model binaries live in Drive or an object/artifact store, referenced by checksums and immutable version identifiers.

What is completed where

Location	Responsibilities	Primary output
Colab GPU	Training, fine-tuning, AMP, CUDA smoke tests, export parity, calibration trials, GPU benchmarks	Checkpoints, ONNX, metrics, benchmark JSON, manifest
VS Code / local CPU	API, ONNX CPU/INT8 inference, unit/integration tests, Redis, PostgreSQL/pgvector, Celery, Docker Compose, MLflow registration	Runnable production stack and test evidence
Target GPU environment	Final TensorRT build, hardware-specific performance validation, GPU serving image	Deployable .plan engine and target benchmark
GitHub	Reviewable code/notebooks, CPU CI, container build, release approval	Traceable commit and release tag
Drive / artifact store	Large data, checkpoints, exported bundles, resumable checkpoints	Durable handoff independent of Colab VM lifetime

Recommended delivery horizon

A production-oriented challenge submission is realistically a five-week sequence for a small team, with model work and platform work overlapping. A compressed three-week delivery is possible only by using pretrained detection/similarity models, limiting training sweeps, and keeping deployment to ONNX CPU while treating TensorRT as a validated target-GPU follow-up. Estimates in this guide are engineering effort, not guarantees.

Target architecture and handoff

The workflow separates compute location from source of truth. The same versioned Python functions are launched from Colab notebooks, local CLI commands, and later CI/CD jobs.

1. Git commit	2. Colab GPU	3. Artifact handoff	4. MLflow gate	5. FastAPI runtime
Code + configs + notebook launcher	Train / export / calibrate / benchmark	Drive or object store + manifest + SHA-256	Register candidate; compare; alias champion	Download immutable bundle; load ONNX CPU or target TensorRT

SERVING ANSWER

The online endpoint is Nginx → FastAPI → runtime adapter. MLflow is the model control plane; it is not the request path. Celery workers use the same inference service/module for batch jobs. Changing a model alias alone does not silently hot-swap a live process: promotion emits a deployment action that downloads, verifies, warms and atomically activates the bundle.

Runtime deployment profiles

Profile	Model runtime	Where used	Promotion requirement
Local / challenge default	ONNX Runtime CPU; static INT8 where accuracy gate passes	Laptop/desktop Docker Compose and reviewer demo	CPU parity, p95 latency, accuracy delta, bundle checksum
Portable GPU	ONNX Runtime CUDA or PyTorch CUDA	Cloud GPU service when TensorRT is not yet hardened	CUDA parity and target-GPU benchmark
Optimized target GPU	TensorRT FP16/INT8	Controlled production GPU node	Rebuild on target; engine load test; accuracy and load benchmark

Model choices retained from the solution blueprint

Task	Baseline	Colab workload	Release evidence
Classification	EfficientNet-B0	Fine-tune on Tiny ImageNet; make dataset/model configurable for CIFAR-100	Top-1/Top-5, calibration, latency
Detection	YOLOX-Tiny	Start from COCO pretrained weights; validate a deterministic COCO subset	mAP, per-class AP, latency
Similarity	OpenCLIP ViT-B/32	Pretrained image encoder; normalized embeddings stored with pgvector	Recall@K, cosine agreement, query latency

Repository boundary

```
notebooks/colab/         # thin, reviewable GPU launchers
pipelines/               # Python entrypoints: train, export, evaluate, package
models/                  # task modules, preprocessing, runtime adapters, registry client
api/                     # FastAPI routers, schemas, lifespan, auth, metrics
workers/                 # Celery tasks for batch inference and maintenance
db/                      # migrations, repositories, pgvector queries
configs/                 # model/data/runtime YAML; no secrets
artifacts/                # local sync target; gitignored
tests/                   # unit, contract, integration, model parity, load fixtures
.github/workflows/        # CPU CI, image release, manual GPU deployment gate
pyproject.toml            # canonical dependencies and tools
uv.lock                   # reproducible dependency resolution
requirements/colab.txt    # exported, pinned Colab installation set
```

Notebook sequence and operating contract

Every notebook is independently restartable, records the Git commit, captures the runtime, writes durable checkpoints, and ends by producing machine-readable outputs. Notebook order is explicit so a teammate can run only the needed stage.

Notebook	Needs	Responsibility	Required output
00_runtime_check.ipynb	GPU	Clone/checkout commit; mount Drive; verify torch.cuda, GPU, disk, secrets; capture environment	environment.json
01_data_prepare.ipynb	CPU/ GPU	Download/verify data; deterministic split; stage archive to VM; create manifests	data_manifest.json + checksums
02_classification_train.ipynb	GPU	Fine-tune EfficientNet-B0 with AMP, checkpoint/resume and evaluation	best.pt + metrics.json
03_detection_prepare.ipynb	GPU	Acquire YOLOX-Tiny pretrained model; deterministic COCO validation; optional fine-tune	detector.pt + COCO metrics
04_similarity_prepare.ipynb	GPU	Prepare OpenCLIP encoder; build normalized embedding sample; Recall@K evaluation	encoder.pt/reference + embeddings sample
05_export_onnx.ipynb	GPU	Export each task; validate graph; compare PyTorch vs ONNX outputs	model.onnx + parity.json
06_int8_quantization.ipynb	CPU/ GPU	Static calibration where supported; accuracy/metric regression check	model.int8.onnx + calibration manifest
07_gpu_benchmark.ipynb	GPU	Warm-up; synchronized latency; throughput and memory evidence	benchmark_gpu.json
08_package_artifacts.ipynb	CPU	Create immutable bundle, model card and SHA-256 checksums; upload handoff	bundle.tar.gz + manifest.json

Rules for every notebook

- First cell prints notebook name, Git commit SHA, UTC start time and selected config file.
- Runtime guard fails immediately if a GPU-required notebook does not see CUDA. A GPU-enabled runtime does not imply the code is actually using the GPU. [S2]
- Install dependencies from a committed, pinned file; capture Python, pip/uv, torch, CUDA, cuDNN, ONNX Runtime and TensorRT versions.
- Use config-driven paths; do not embed a teammate's Drive path or machine-specific absolute paths.

- Copy archives from Drive to the Colab VM before high-volume training I/O; copy only completed checkpoints and final bundles back. Google documents that many small Drive I/O operations can be slow or quota-sensitive. [S2]
- Write a checkpoint at a predictable interval and on graceful interruption; resume only when config hash, dataset hash and code SHA match.
- Never store access tokens, API keys, database passwords or signed URLs in notebook cells or outputs.
- Clear bulky outputs before Git review; retain only short evidence cells and links/IDs to durable artifacts.

Minimal Colab bootstrap

```
# 1) Runtime > Change runtime type > GPU
!nvidia-smi
import torch, sys
assert torch.cuda.is_available(), "GPU-required notebook: select a GPU runtime"
print(sys.version)
print(torch.__version__, torch.version.cuda, torch.cuda.get_device_name(0))

# 2) Clone and pin execution to a known commit/branch
!git clone https://github.com/<org>/<repo>.git /content/ml-platform
%cd /content/ml-platform
!git rev-parse HEAD

# 3) Install only from committed constraints
!python -m pip install -r requirements/colab.txt

# 4) Mount durable checkpoint/artifact storage only after reviewing notebook code
from google.colab import drive
drive.mount('/content/drive')
```

SECURITY NOTE

Mounting Drive allows notebook code to access mounted files. Review code before granting access, isolate project artifacts in a dedicated folder, and never expose secrets in saved output. [S2]

Artifact contract and Colab → VS Code handoff

A model is not handoff-ready because a checkpoint exists. The unit of transfer is an immutable, checksum-verified bundle that contains enough information to reproduce preprocessing, evaluate quality and load a supported runtime.

Bundle layout

```
artifacts/<task>/<model>/<semantic-version>/
├── checkpoint.pt             # training recovery; not necessarily served
├── model.onnx                # portable inference graph
├── model.int8.onnx           # optional; only when accuracy gate passes
├── model.plan                # optional demo only; rebuild for target GPU
├── preprocessing.json        # resize, crop, normalization, color order
├── labels.json               # class/index mapping or category metadata
├── metrics.json              # quality metrics and dataset split ID
├── parity.json               # PyTorch ↔ ONNX/runtime comparison
├── benchmark.json            # hardware + runtime + latency/throughput
├── manifest.json             # identity, commit, dependencies, files, hashes
├── MODEL_CARD.md             # intended use, limitations, evaluation
└── checksums.sha256          # tamper/corruption detection
```

Mandatory manifest fields

Section	Required fields
identity	task, model_name, model_version, created_at_utc, run_id, Git SHA
training	dataset_id/hash, split hash, seed, config hash, base weights and license
environment	Python, torch, CUDA/cuDNN, ONNX/opset, ONNX Runtime, TensorRT, GPU model
pre/post-processing	input tensor name/shape/dtype, normalization, output schema, thresholds, NMS settings
quality	primary metric values, acceptance thresholds, regression delta, calibration details
performance	warm-up count, repetitions, batch sizes, synchronization method, p50/p95/p99, throughput
integrity	relative file paths, size, SHA-256, license/provenance metadata
compatibility	supported runtimes/devices; TensorRT target architecture and build flags

Handoff procedure

- Colab writes intermediate checkpoints to a run-scoped Drive folder and final output to a temporary staging folder.
- Packaging code calculates checksums, validates required files, archives the bundle and then performs an atomic rename/move to the completed folder.

- A teammate syncs the bundle to local artifacts/incoming/ using Drive download, rclone, or a team artifact-store client.
- Local import verifies checksums and manifest schema before any model deserialization or registry operation.
- CPU parity and API contract tests run locally. Failed bundles remain candidates and cannot receive a champion alias.
- The accepted bundle is registered in MLflow with its immutable source URI and build metadata; promotion produces a deployment event/release, not a silent alias-only switch.

RECOVERY PRINCIPLE

Colab VMs are disposable. Anything that cannot be recreated cheaply—checkpoints, metrics, manifests, exported graphs—must leave /content before the session ends. Colab states that VMs are deleted after idle periods and have a maximum lifetime. [S2]

VS Code import and local CPU operating procedure

Colab notebooks use the open Jupyter .ipynb format and can be opened natively in VS Code through the Jupyter extension. The local kernel is intentionally CPU-only; GPU sections must fail clearly or be skipped through explicit capability guards. [S2][S3]

One-time setup

- Install current VS Code, the Microsoft Python extension and the Microsoft Jupyter extension.
- Clone the repository, create .venv, select it with Python: Select Interpreter, then select the same environment as the notebook kernel.
- Install the locked local dependency group. Do not install CUDA-only packages into the CPU environment unless the resolver explicitly supports them.
- Download one accepted model bundle into artifacts/models/<task>/<version>/ and run checksum verification before loading it.
- Start PostgreSQL/pgvector, Redis, MLflow, API, worker, Prometheus and Grafana through Docker Compose profiles.

```
git clone https://github.com/<org>/<repo>.git
cd <repo>
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
python -m pip install -U pip
python -m pip install -e ".[dev,serving]"
python -m ipykernel install --user --name ml-challenge-local

python -m pipelines.artifacts.verify artifacts/incoming/<bundle>.tar.gz
docker compose --profile observability up -d
pytest -m "not gpu" -q
```

Notebook import discipline

Cell class	Colab behavior	VS Code local behavior
Environment/bootstrap	Runs Colab install, Drive mount and CUDA guard	Uses active .venv; does not mount Drive
Reusable pipeline call	Calls the same pipelines.* Python function	Runs on tiny CPU fixture or dry-run mode
GPU-only training	Runs when device=cuda	Skipped with an explicit message; never silently falls back for long training
Evaluation/parity	Runs full GPU evaluation	Runs deterministic small CPU contract set
Visualization	May retain compact charts	Re-runs from metrics JSON without needing model training

Useful capability guard

```
from models.runtime import DeviceCapability

cap = DeviceCapability.detect()
if not cap.cuda_available:
    raise RuntimeError(
        "This notebook stage requires CUDA. Run it in Google Colab; "
        "use --dry-run or the CPU smoke notebook locally."
    )
```

DO NOT DUPLICATE LOGIC

If a bug fix is made only inside a notebook cell, the project is not production-ready. Move it into a tested module, import the module from the notebook, then commit both the code and any notebook explanation/output changes.

MLflow modes, promotion and serving

The implementation supports two tracking modes without changing training code. Start with the disconnected handoff when networking/authentication is a distraction; move to a shared tracking server when the team needs live experiment visibility.

Mode	Colab behavior	Local/team behavior	When to choose
A — disconnected handoff (recommended first)	Logs to local files, creates bundle + metrics + manifest, uploads to Drive/object store	Verification script creates/imports MLflow run and registers the candidate	Challenge build, no public MLflow endpoint, simplest security boundary
B — remote MLflow	Uses secret tracking URI/token and logs directly to shared server/object storage	Reviewers compare runs and promote registered versions centrally	Stable team environment with TLS, auth, database backend and durable artifacts

Promotion state machine

State	Required checks	Allowed action
experiment	Training completed; config and environment captured	Compare runs; no serving
candidate	Bundle schema + checksum + offline metrics + parity pass	Deploy to staging/shadow
challenger	Staging integration, security and performance pass	Receive controlled A/B traffic
champion	A/B or review approval, rollback bundle retained	Build release and activate immutable bundle
rejected/archived	Failed gate or superseded	Retain evidence; never serve

Deployment action after promotion

- Resolve the champion alias to a concrete registered model version and immutable artifact URI. MLflow aliases are mutable references intended for lifecycle/deployment workflows. [S7]
- Download to a versioned staging directory; verify SHA-256, manifest schema, supported runtime and dependency compatibility.
- Load and warm the runtime in a separate process or inactive model slot; execute health, golden-image and latency smoke tests.
- Atomically switch the active model reference or roll the deployment; expose model_version in every response and metric label.
- If any post-switch SLO or error-rate gate fails, restore the previous immutable bundle and alias/release metadata.

WHY MLFLOW DOES NOT SERVE REQUESTS HERE

MLflow remains valuable for tracking and registry, but the challenge needs custom preprocessing, three task schemas, Redis behavior, Celery batch execution, PostgreSQL/pgvector integration, rate limiting, A/B routing and Prometheus labels. A custom FastAPI service is the cleaner request path.

Indicative five-week delivery map

The phase gates are sequential, but platform and model tracks can overlap. The schedule below assumes two to three contributors and deliberately includes validation time.

Window	Model/GPU track	Platform/local track	Review gate
Week 1	Runtime check, data manifests, classification baseline	Repository, configs, local Compose skeleton, CI lint/tests	Data and environment reproducibility
Week 2	Classification training; detection/similarity preparation	Runtime interfaces, schema contracts, database migrations	All three task metrics recorded
Week 3	ONNX export, quantization, GPU benchmark, packaging	ONNX CPU parity, artifact import, MLflow registry	Candidate bundles registered
Week 4	Targeted re-runs only	FastAPI, Celery, Redis, pgvector, integration and load tests	End-to-end staging pass
Week 5	Target GPU TensorRT build if GPU deployment exists	Observability, A/B, drift, security, release/rollback	Release candidate and demo

Critical path

- Data manifest → trained/evaluated baseline → ONNX parity → immutable bundle → API integration → release gate.

- TensorRT target deployment is a separate branch after portable ONNX succeeds; it must not block a CPU-serving challenge demo.
- Observability and security start with the service skeleton; they are not end-of-project decorations.

Team responsibility split

Role	Primary ownership	Must review
ML engineer	Data/model configs, training, export, parity, metrics, model card	Runtime adapter and preprocessing contract
Backend engineer	FastAPI schemas, runtime orchestration, Celery, Redis, PostgreSQL/pgvector	Model input/output and batch semantics
Platform engineer	Docker, MLflow, CI/CD, target GPU build, monitoring, security	Artifact contract, deployment/rollback
QA/reviewer	Golden set, integration/load tests, acceptance evidence, demo runbook	Every phase gate and release decision

PHASE 00

Freeze architecture, scope and acceptance gates

Execution location	VS Code / team review
Indicative effort	0.5–1 day
Exit outcome	Signed decision record and executable acceptance matrix

Objective

Turn the challenge into measurable contracts before any GPU time is consumed. Define the serving profile, model baselines, dataset splits, runtime matrix, security boundary and release criteria.

Implementation work

- Create docs/adr/0001-serving-and-colab-boundary.md: Colab executes GPU development; FastAPI serves; MLflow tracks and governs; GitHub Actions orchestrates automated CI/CD.
- Define input/output JSON schemas for classification, detection and similarity, including error codes, request IDs, model version and processing time.
- Freeze primary quality metrics and preliminary gates: classification Top-1/Top-5, detection mAP, similarity Recall@K; define maximum accepted runtime delta after export/quantization.
- Define target profiles: local ONNX CPU, optional ONNX CUDA, final TensorRT target GPU. Record that Colab GPU type is not guaranteed. [S2]
- Choose dataset split IDs and seed policy. Separate calibration, validation, test and drift-reference sets to prevent leakage.
- Document licenses/provenance for pretrained weights and datasets before redistribution.

Technology and repository touchpoints

Area	Decision / files
Architecture	docs/adr/0001-serving-and-colab-boundary.md; docs/architecture.md
Contracts	api/schemas/*.py; docs/api-contract.md; configs/acceptance.yaml
Configuration	configs/models/*.yaml; configs/data/*.yaml; configs/runtime/*.yaml
Governance	MODEL_CARD template, SECURITY.md, data/model license inventory

Deliverables

- Acceptance matrix with numeric or review-required gates
- Architecture diagram and runtime responsibility matrix
- Initial risk register and licensing record

PHASE GATE

All contributors can answer where training, artifacts, registry, serving, batch execution and final TensorRT compilation occur; no unresolved API or dataset ambiguity remains.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: treating 'production-ready' as a collection of tools. Control: every technology must map to an explicit contract, owner, artifact and test.

PHASE 01

Build the repository and reproducible environments

Execution location	VS Code first; validate in Colab
Indicative effort	1 day
Exit outcome	Local CPU stack and Colab bootstrap reproduce from one commit

Objective

Create a package-first repository in which notebooks, CLI commands, API processes and workers import the same tested modules.

Implementation work

- Create pyproject.toml with separate dev, serving, model and observability dependency groups; resolve a lock and export requirements/colab.txt.
- Pin direct dependencies and record compatibility across Python, torch/torchvision, CUDA wheels, ONNX/opset, ONNX Runtime and TensorRT.
- Add ruff, mypy, pytest, coverage and pre-commit; configure deterministic seeds and structured logging.
- Create Docker Compose services for api, worker, redis, postgres/pgvector, mlflow, prometheus and grafana. Use CPU runtime by default.
- Add 00_runtime_check.ipynb and a CLI-equivalent environment report so Colab and local diagnostics produce the same JSON schema.
- Gitignore Drive sync folders, datasets, checkpoints, model binaries, secrets and notebook checkpoints; permit only small test fixtures.

Technology and repository touchpoints

Area	Decision / files
Packaging	pyproject.toml, uv.lock, requirements/colab.txt, requirements/serving.txt
Quality	ruff, mypy, pytest, pre-commit, coverage

Area	Decision / files
Runtime	Dockerfile.api, Dockerfile.worker, compose.yaml, .env.example
Diagnostics	pipelines/environment_report.py; notebooks/colab/00_runtime_check.ipynb

Operator command pattern

```
python -m pipelines.environment_report --output artifacts/environment-local.json
docker compose config
docker compose up -d postgres redis mlflow
pytest tests/unit -q
```

Deliverables

- Clean local installation from lock
- Colab environment.json tied to Git SHA
- CPU Compose services become healthy
- CI lint and unit-test skeleton passes

PHASE GATE

A teammate can clone the repository, create the CPU environment, run tests and start infrastructure; Colab can check out the same SHA and complete its runtime check.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: Colab succeeds because of unrecorded preinstalled packages. Control: explicit installation, full environment capture and clean-runtime smoke test.

PHASE 02

Create deterministic data pipelines and manifests

Execution location	Colab for acquisition/staging; VS Code for code/tests
Indicative effort	1–2 days
Exit outcome	Immutable split and calibration manifests for all three tasks

Objective

Make data preparation reproducible without committing large datasets. Dataset identity, preprocessing and split membership must be reconstructable from manifests.

Implementation work

- Implement download adapters with source URL/version, license, expected archive hash and retry-safe extraction.
- Generate deterministic train/validation/test/calibration manifests using stable sample IDs and a fixed seed. Never calibrate INT8 on the test set.
- For Tiny ImageNet, validate class count, image counts, corrupt images and label mapping; persist labels.json.
- For COCO, select a deterministic subset manifest and preserve original annotations/category IDs. Do not randomly change the validation subset between runs.
- For similarity, define a query/gallery split and positive-pair ground truth; normalize image decoding and RGB conversion consistently.
- Archive large datasets for Drive transfer, copy the archive to /content, verify checksum, extract to Colab ephemeral disk and train from local VM storage.
- Add data unit tests using tiny synthetic fixtures: invalid image, missing label, duplicate ID, corrupt archive and split leakage.

Technology and repository touchpoints

Area	Decision / files
Data code	pipelines/data/*.py; models/common/preprocessing.py
Manifests	data/manifests/<dataset>/<version>/*.jsonl; committed if small

Area	Decision / files
Validation	Great Expectations is optional; core checks remain plain Python + pytest
Storage	Drive/object store for archives; Git for metadata and hashes

Deliverables

- Data quality report per task
- Immutable split/calibration manifests
- labels.json/category mapping
- Dataset checksum and provenance record

PHASE GATE

Re-running preparation with the same source, code, config and seed yields identical manifest hashes; no sample exists in conflicting splits.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: training directly against thousands of Drive files causes slow or quota-sensitive I/O. Control: archive, copy once to the VM, verify, extract locally, write only checkpoints/results back. [S2]

PHASE 03

Fine-tune and validate EfficientNet-B0 classification

Execution location	Google Colab GPU
Indicative effort	2–3 days
Exit outcome	Candidate classifier checkpoint with reproducible quality evidence

Objective

Produce the only task-specific fine-tuning run required for the primary baseline while designing the trainer to generalize to CIFAR-100 or another configured dataset.

Implementation work

- Load ImageNet-pretrained EfficientNet-B0 through timm/torchvision and replace the classification head using config-defined class count.
- Use train-only augmentations; make validation preprocessing identical to serving preprocessing and serialize it into preprocessing.json.
- Train with torch.amp autocast and GradScaler on CUDA; keep FP32 master weights and confirm the loss is finite. PyTorch AMP combines lower precision execution with gradient scaling. [S4]
- Implement checkpoint/resume containing model, optimizer, scheduler, scaler, epoch, best metric, random states, config hash, dataset hash and Git SHA.
- Use early stopping and a small, declared hyperparameter search budget. Do not spend Colab quota on an unbounded sweep.
- Report Top-1, Top-5, per-class recall, confusion summary, Expected Calibration Error and representative failure images.
- Run a deterministic inference snapshot on golden images for later PyTorch/ONNX/API parity tests.

Technology and repository touchpoints

Area	Decision / files
Model	models/classification/model.py; EfficientNet-B0

Area	Decision / files
Training	pipelines/train_classification.py; torch.amp; AdamW/SGD; scheduler
Config	configs/models/efficientnet_b0_tiny_imagenet.yaml
Notebook	notebooks/colab/02_classification_train.ipynb

Operator command pattern

```
python -m pipelines.train_classification \
  --config configs/models/efficientnet_b0_tiny_imagenet.yaml \
  --device cuda --amp --resume auto \
  --output /content/drive/MyDrive/ml-challenge/runs/classification/<run-id>
```

Deliverables

- best.pt and last.pt
- metrics.json and confusion/error artifacts
- golden_outputs.npz
- preprocessing.json and labels.json
- MLflow-compatible run folder

PHASE GATE

Best checkpoint beats the agreed baseline, has no leakage, resumes from a simulated interruption and reproduces golden outputs within tolerance.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: a Colab termination loses progress or a resume silently changes the experiment. Control: periodic durable checkpoints and strict resume validation against config/data/code hashes.

PHASE 04

Prepare and validate YOLOX-Tiny detection

Execution location	Google Colab GPU
Indicative effort	1–2 days
Exit outcome	COCO-pretrained detector candidate with deterministic evaluation

Objective

Prioritize production integration and validation over unnecessary retraining. Use pretrained COCO weights unless the challenge dataset requires adaptation.

Implementation work

- Pin the YOLOX code/weights version and record the original model license and download checksum.
- Wrap preprocessing, output decoding and NMS behind the task interface; explicitly record letterboxing, confidence threshold, IoU threshold and coordinate convention.
- Evaluate on the deterministic COCO subset and report COCO mAP, AP50, AP75, per-class/size metrics and images with common false positives/negatives.
- Create golden detections that include zero detections, multiple overlapping boxes, small objects and boundary boxes.
- Only fine-tune if a stated dataset-domain gap exists; otherwise preserve time for export, API and system reliability.
- Verify that postprocessing is deterministic and shared between PyTorch and exported runtimes or is explicitly implemented in the API layer.

Technology and repository touchpoints

Area	Decision / files
Model	models/detection/yolox_tiny.py; pinned pretrained checkpoint
Evaluation	pipelines/evaluate_detection.py; pycocotools
Config	configs/models/yolox_tiny_coco.yaml; configs/data/coco_subset.yaml
Notebook	notebooks/colab/03_detection_prepare.ipynb

Deliverables

- detector checkpoint/reference manifest
- COCO metric JSON
- threshold/NMS preprocessing contract
- golden detection set
- error analysis contact sheet

PHASE GATE

Deterministic subset evaluation passes the baseline; decoded boxes satisfy schema and coordinate tests; PyTorch predictions are captured for export parity.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: ONNX output looks different because NMS or letterboxing differs, not because the network differs. Control: separately test preprocessing, raw logits/boxes and postprocessing contracts.

PHASE 05

Prepare OpenCLIP similarity and the vector contract

Execution location	Google Colab GPU for embedding; VS Code/PostgreSQL for retrieval
Indicative effort	1–2 days
Exit outcome	Validated embedding model and pgvector-compatible retrieval contract

Objective

Create a similarity model whose embedding normalization, distance function and index behavior are explicit and testable from GPU encoding through database retrieval.

Implementation work

- Pin OpenCLIP ViT-B/32 architecture, pretrained tag, weight checksum and license/provenance.
- Serialize the exact image preprocessing and embedding dimension; L2-normalize embeddings before cosine/IP search.
- Build query/gallery manifests and report Recall@1/5/10, mean reciprocal rank and representative nearest-neighbor panels.
- Export a deterministic embedding sample with image IDs for local pgvector ingestion and retrieval contract tests.
- Create PostgreSQL schema with vector dimension constraint, model_version, source image ID, metadata and timestamps; use a version-aware index/query.
- Define re-embedding migration: new model version writes a new namespace/index and swaps after backfill/validation, never corrupting the active gallery.

Technology and repository touchpoints

Area	Decision / files
Model	models/similarity/openclip_vit_b32.py; open_clip_torch
Evaluation	pipelines/evaluate_similarity.py; Recall@K/MRR
Database	PostgreSQL + pgvector; db/migrations/*; db/similarity_repository.py

Area	Decision / files
Notebook	notebooks/colab/04_similarity_prepare.ipynb

Deliverables

- Embedding model reference/checkpoint
- preprocessing.json + vector dimension
- query/gallery metrics
- sample_embeddings.parquet/npz
- pgvector schema and retrieval tests

PHASE GATE

GPU and CPU embeddings meet cosine agreement tolerance; pgvector Top-K matches an in-memory exact-search oracle on the test fixture.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: mixing embeddings from different model versions produces meaningless neighbors. Control: model_version is part of storage, query, cache and migration keys.

PHASE 06

Export ONNX, quantize INT8 and validate runtime parity

Execution location	Colab for export/calibration; VS Code for ONNX CPU verification
Indicative effort	1–2 days
Exit outcome	Portable ONNX bundles with accepted quality/performance trade-offs

Objective

Make ONNX the portable serving contract. Quantization is a candidate optimization that is promoted only when task metrics remain within the agreed regression budget.

Implementation work

- Export with named inputs/outputs, declared opset, eval mode and dynamic batch axes only where supported; validate the ONNX graph and shape inference. PyTorch provides torch.onnx export for this conversion. [S5]
- Compare raw outputs and final task outputs against PyTorch on golden, random and boundary samples; use task-aware tolerances instead of one universal threshold.
- Use static INT8 quantization for CNN workloads when representative calibration data exists; ONNX Runtime maps floating-point values into 8-bit quantized space. [S6]
- Keep calibration data representative, isolated from the test set and identified by manifest hash; test per-channel options where supported.
- Measure classification metric delta, detection mAP delta and similarity Recall@K/cosine delta separately; reject INT8 per model if quality loss is excessive.
- Verify the resulting graphs on local ONNX Runtime CPU. Do not claim an optimization based only on model file size.
- Treat TensorRT export/build as a separate target-specific operation. Preserve ONNX as the reconstructable source artifact.

Technology and repository touchpoints

Area	Decision / files
Export	models/export/onnx_export.py; onnx; torch.onnx
Quantization	models/quantization/onnx_int8.py; onnxruntime.quantization
Parity	tests/model_parity/*; pipelines/validate_runtime.py
Notebooks	05_export_onnx.ipynb; 06_int8_quantization.ipynb

Deliverables

- FP32 ONNX for each task
- Accepted INT8 ONNX where eligible
- parity.json and quantization_report.json
- calibration manifest
- runtime compatibility metadata

PHASE GATE

Every exported candidate loads in local ONNX Runtime CPU, matches task output contracts and passes quality regression gates; unsupported INT8 candidates fall back to FP32 without blocking release.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: quantization succeeds technically but degrades recall/mAP. Control: per-task acceptance gates and independent promotion of FP32 versus INT8 artifacts.

PHASE 07

Benchmark fairly and package immutable bundles

Execution location	Colab GPU plus VS Code CPU
Indicative effort	1 day
Exit outcome	Comparable CPU/GPU evidence and complete artifact bundles

Objective

Produce benchmark results that describe their hardware and method and can be compared across runtime candidates without misleading timing.

Implementation work

- Define fixed benchmark inputs and batch sizes; separate preprocessing, model inference, postprocessing and end-to-end latency.
- Warm each runtime, synchronize CUDA before/after timing, discard warm-up measurements and report p50/p95/p99, throughput, peak memory and cold-start/model-load time.
- Record GPU/CPU model, RAM/VRAM, OS, Python, runtime versions, threads, precision, dynamic/static shapes and power mode when known.
- Run local CPU FP32 and INT8 benchmarks using the same logical inputs; report results as different hardware profiles rather than a single winner.
- Build each bundle from a clean staging directory, validate mandatory files, generate checksums and write a complete model card.
- Upload only after local packaging validation completes; use completed/ and incomplete/ prefixes to prevent partial bundles from being registered.

Technology and repository touchpoints

Area	Decision / files
Benchmark	pipelines/benchmark.py; torch.cuda synchronization; ONNX Runtime session settings
Packaging	pipelines/artifacts/package.py; JSON Schema/Pydantic manifest
Notebook	07_gpu_benchmark.ipynb; 08_package_artifacts.ipynb
Evidence	benchmark_cpu.json, benchmark_gpu.json, MODEL_CARD.md

Deliverables

- Hardware-qualified benchmark JSON
- Immutable bundles for all tasks
- checksums.sha256
- Model cards with known limitations
- Artifact inventory table

PHASE GATE

A fresh local environment can verify and load every bundle; benchmark methodology is documented and results are tagged with exact hardware/runtime identity.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: reporting Colab GPU latency as expected production latency. Control: label it development evidence; repeat final performance testing on the deployment node.

PHASE 08

Register models and implement promotion controls

Execution location	VS Code/local MLflow first; remote MLflow optional
Indicative effort	1 day
Exit outcome	Versioned candidates, aliases and auditable deployment events

Objective

Bring disconnected Colab artifacts into a model lifecycle system without making the registry the online serving endpoint.

Implementation work

- Run MLflow with a database-backed backend store for registry use; use a dedicated PostgreSQL database/schema and durable artifact volume or object store.
- Import the Colab run metadata, metrics, parameters and bundle URI while preserving original run_id/Git SHA in tags.
- Register task-specific models and apply descriptive tags: dataset_hash, runtime, precision, quality gate, license and security scan status.
- Use candidate/challenger/champion aliases; require an approval record and immutable bundle URI before champion assignment. [S7]
- Create a promotion CLI/API that re-validates artifacts and emits a deployment request. Keep alias mutation and runtime activation separate but correlated.
- Implement rollback that resolves and activates the previous version without rebuilding it.

Technology and repository touchpoints

Area	Decision / files
Tracking	MLflow server; PostgreSQL backend; local volume or S3-compatible artifacts
Registry	models/registry/mlflow_client.py; pipelines/register_model.py
Promotion	pipelines/promote_model.py; approvals and audit log

Area	Decision / files
Config	MLFLOW_TRACKING_URI via environment/secret; never notebook source

Deliverables

- Registered candidate versions
- Champion/challenger alias policy
- Promotion and rollback commands
- Audit record linking model, Git SHA, tests and deployment

PHASE GATE

A candidate cannot become champion without checksum, metric, parity and integration evidence; rollback to the prior bundle is rehearsed.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: exposing an unauthenticated MLflow server to Colab/public internet. Control: start in disconnected mode or use TLS, authentication, least privilege and restricted network access.

PHASE 09

Build the FastAPI inference service and runtime adapters

Execution location	VS Code / local CPU
Indicative effort	2–3 days
Exit outcome	Stable synchronous API serving all three tasks on ONNX CPU

Objective

Create a single production service boundary with task-specific schemas and interchangeable runtime adapters, loading models once during application lifespan.

Implementation work

- Implement RuntimeAdapter interfaces for PyTorch, ONNX Runtime and TensorRT with load, warmup, predict, health and metadata methods.
- Use FastAPI lifespan to resolve, verify, load and warm model bundles before readiness becomes true; release resources on shutdown. FastAPI recommends lifespan for startup/shutdown logic such as shared ML models. [S9]
- Create endpoints: POST /v1/classify, /v1/detect, /v1/similar and batch submission/status endpoints; add /livez, /readyz and /metrics.
- Validate MIME type, decoded image format, pixel dimensions, payload bytes, batch size and timeouts; protect image decoding against decompression bombs and malformed files.
- Return request_id, task, model_name, model_version, runtime, processing_ms and typed predictions; never leak stack traces or filesystem paths.
- Use bounded concurrency and worker sizing based on runtime threads/memory. Avoid loading a full model copy per request.
- Add golden-image API tests and failure contracts for invalid input, unavailable model and timeout.

Technology and repository touchpoints

Area	Decision / files
API	FastAPI, Pydantic v2, Uvicorn/Gunicorn, python-multipart/Pillow

Area	Decision / files
Lifecycle	api/main.py; api/lifespan.py; model manager
Runtimes	models/runtimes/{onnx,pytorch,tensorrt}.py
Schemas	api/schemas/{classification,detection,similarity,common}.py

Operator command pattern

```
MODEL_RUNTIME=onnx MODEL_DEVICE=cpu \
MODEL_BUNDLE_ROOT=artifacts/models \
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Deliverables

- OpenAPI schema and example requests
- ONNX CPU inference for all tasks
- Health/readiness/metrics endpoints
- Golden API and negative tests
- Runtime/model metadata in responses

PHASE GATE

The service starts from immutable local bundles, becomes ready only after warmup, returns schema-valid predictions and survives malformed/oversized request tests.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: model load occurs on every request or each process loads uncontrolled copies. Control: lifespan model manager, explicit worker count, memory benchmark and readiness gate.

PHASE 10

Add Celery, Redis, PostgreSQL and pgvector workflows

Execution location	VS Code / Docker Compose
Indicative effort	2 days
Exit outcome	Reliable async batch and similarity persistence with idempotency

Objective

Move long-running/batch work out of request workers, introduce cache/rate-limit controls, and make persistence and job state explicit.

Implementation work

- Use Redis as Celery broker/result backend for the challenge and as a TTL cache/rate-limit store with namespaced keys containing task, model_version, preprocessing_version and input hash.
- Define Celery queues by task/resource profile; bound retries with exponential backoff and jitter; route permanent validation errors directly to failure.
- Make batch jobs idempotent using client/job idempotency keys; store job state and result metadata in PostgreSQL, not only Redis.
- Store similarity embeddings in pgvector with model_version and source identity; use exact search for small challenge data and introduce HNSW/IVFFlat only after benchmark.
- Create Alembic migrations for jobs, requests, model deployments, A/B assignments, embeddings and drift summaries.
- Define retention/cleanup for raw uploads, results, Redis keys and embeddings; do not retain user images indefinitely by accident.
- Add integration tests for worker retry, duplicate submission, worker crash, stale result and database transaction rollback.

Technology and repository touchpoints

Area	Decision / files
Async	Celery; workers/tasks.py; queue routing and retry policy

Area	Decision / files
Cache/broker	Redis; versioned cache keys; token-bucket/sliding-window rate limiting
Database	PostgreSQL + pgvector; SQLAlchemy 2; Alembic
Testing	Docker Compose integration profile; pytest; fake small models

Deliverables

- Async batch API and worker
- Persistent job/audit schema
- pgvector Top-K retrieval
- Rate limiting and cache metrics
- Failure/retry integration evidence

PHASE GATE

Duplicate job submission is safe, failed tasks are observable/retry-bounded, results survive Redis eviction where required, and similarity retrieval passes the exact-search oracle.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: Redis becomes a single unbounded store for broker, cache and durable truth. Control: separate key namespaces/DBs, TTLs and memory policy; PostgreSQL owns durable state.

PHASE 11

Implement model validation, A/B testing and drift controls

Execution location	VS Code / staging; Colab only for approved retraining
Indicative effort	2 days
Exit outcome	Automated evidence for model and data behavior after deployment

Objective

Detect regressions and population changes without requiring ground-truth labels for every request, while preserving a path to supervised quality checks.

Implementation work

- Route champion/challenger by a stable hash of authenticated user/tenant/request key; persist assignment to prevent cross-version flip-flopping.
- Start with shadow mode when safe: challenger receives copied inputs but does not affect user output; compare prediction divergence and runtime overhead.
- Define online metrics by model version and route: request count, error rate, p95 latency, timeouts, cache hit rate, queue depth and task-specific output distributions.
- Build drift reference profiles from the approved validation/training sample: image dimensions, brightness/color histograms, embedding summaries, class confidence and detection counts.
- Use PSI/KS or embedding-distance indicators as alerts, not proof of quality loss; investigate and sample labels before retraining.
- Create scheduled drift reports and retraining proposals. The production scheduler is GitHub Actions/Celery/another durable orchestrator—not a Colab notebook session.
- Promotion gates include minimum traffic/sample size, no SLO regression, acceptable quality proxy/divergence, and human approval.

Technology and repository touchpoints

Area	Decision / files
A/B	Stable hash router; PostgreSQL assignment/audit; version-labeled metrics

Area	Decision / files
Drift	pipelines/monitoring/drift.py; reference profiles; optional Evidently report
Metrics	prometheus_client; Grafana panels and alerts
Retraining	workflow dispatch creates run request; a person launches Colab or target GPU job

Deliverables

- Shadow/A-B router and assignment audit
- Drift reference profile and scheduled report
- Per-version dashboard
- Promotion/rollback decision template

PHASE GATE

The same requester is routed consistently; metrics separate champion/challenger; a simulated drift/SLO breach generates an alert and blocks promotion.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: automatically retraining/promoting on an unlabeled drift signal. Control: drift triggers investigation and a gated pipeline, never autonomous production promotion.

PHASE 12

Harden containers, security and observability

Execution location	VS Code / Docker Compose / staging
Indicative effort	1–2 days
Exit outcome	Reviewable, observable and least-privilege application stack

Objective

Package the service for repeatable deployment and make failure visible before the final release run.

Implementation work

- Use multi-stage CPU API/worker images with pinned digest bases, non-root user, read-only root filesystem where practical, health checks and resource limits.
- Keep a separate target-GPU image based on a compatible NVIDIA CUDA/TensorRT runtime. Do not inflate the CPU image with GPU libraries.
- Place Nginx or an ingress gateway in front of FastAPI for TLS termination, body limits, timeouts and basic rate protection; keep Uvicorn internal.
- Use environment/secret injection; scan source, dependencies and images; generate an SBOM and record scan results against the release.
- Expose Prometheus metrics with bounded label cardinality. Build Grafana dashboards for API SLOs, model/runtime versions, Celery queues, Redis/PostgreSQL health and drift/A-B.
- Use structured JSON logs with request_id/job_id/model_version and redact secrets/PII; define retention and sampling.
- Exercise dependency failure: Redis unavailable, DB unavailable, corrupt model bundle, slow model, worker loss and disk pressure.

Technology and repository touchpoints

Area	Decision / files
Containers	Docker, Compose profiles, separate CPU/GPU Dockerfiles; Docker Compose defines multi-container apps in YAML. [S10]
Gateway	Nginx; TLS, payload limits, timeouts, request IDs

Area	Decision / files
Security	Trivy, pip-audit, Bandit, secret scan, SBOM; non-root runtime
Observability	Prometheus, Grafana, structured logs; alert rules and runbooks

Deliverables

- CPU and GPU image definitions
- Compose stack and health checks
- Security/SBOM reports
- Grafana dashboards and alert rules
- Failure-mode runbook

PHASE GATE

A clean machine starts the CPU profile; health/readiness are correct during dependency failures; no critical unaccepted vulnerability or secret is present; dashboards identify the active model version.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: 'works on my machine' images with root execution and floating dependencies. Control: locked builds, digest pinning, non-root runtime, CI scan and clean-machine test.

PHASE 13

Create CI/CD, release, rollback and the final target-GPU branch

Execution location	GitHub Actions + local/staging + target GPU
Indicative effort	1–2 days
Exit outcome	Tagged release with CPU serving and optional target-GPU TensorRT deployment

Objective

Automate repeatable checks while keeping scarce GPU execution and production promotion gated and hardware-aware.

Implementation work

- On pull requests run formatting, lint, typing, unit tests, notebook structure checks, manifest schema tests, small ONNX CPU parity tests and dependency/secret scans.
- On main/tag build and scan images, produce SBOM/provenance, run Compose integration tests and publish immutable image digests.
- Use workflow_dispatch for model registration/promotion and target-GPU deployment, requiring model version, artifact checksum, approval environment and rollback version.
- Do not make Colab a GitHub Actions runner. GPU CI requires a managed GPU runner, target self-hosted runner or manual evidence import from an approved Colab run.
- On the target GPU, download the accepted ONNX, build TensorRT FP16/INT8 with recorded flags, run engine load/parity/benchmark tests and package the hardware-specific engine.
- NVIDIA documents that serialized TensorRT engines are not generally portable across platforms and, without hardware compatibility mode, across GPU architectures. [S8]
- Deploy canary/shadow first, monitor SLOs, promote gradually, and retain the previous bundle/image for one-command rollback.
- Create a reviewer demo script that starts the CPU stack, submits all task requests and a batch job, shows MLflow/Grafana, simulates promotion, and performs rollback.

Technology and repository touchpoints

Area	Decision / files
CI	.github/workflows/ci.yml; CPU-fast and integration jobs
Release	.github/workflows/release.yml; immutable image tags/digests; SBOM
Promotion	.github/workflows/promote-model.yml; protected environment approval
GPU	.github/workflows/deploy-gpu.yml; target runner; trtexec/build script
Operations	scripts/demo.sh; docs/runbooks/deploy.md and rollback.md

Operator command pattern

```
# CPU release gate
docker compose --profile observability up -d --build
pytest tests/integration tests/model_parity -m "not gpu" -q

# Target GPU gate (executed on the target runner/node)
nvidia-smi
python -m pipelines.build_tensorrt --manifest artifacts/<task>/<version>/manifest.json
python -m pipelines.validate_runtime --runtime tensorrt --golden tests/golden/<task>
```

Deliverables

- Passing CI and image scans
- Immutable release tag/image digest
- CPU end-to-end demo evidence
- Optional target TensorRT engine and benchmark
- Tested promotion and rollback runbooks

PHASE GATE

A clean release is reproducible from a Git tag; CPU service handles all task contracts; rollback is rehearsed; any TensorRT artifact is built and validated on its declared target hardware.

PRIMARY FAILURE MODE AND CONTROL

Failure mode: a Colab-built .plan is copied to a different GPU and fails or produces unsupported behavior. Control: deploy ONNX portably and rebuild TensorRT on the target under a compatibility-tested image.

Definition of done

The project is complete only when another engineer can reproduce, operate and roll it back. Use the following checklist as the submission/release gate.

Repository and reproducibility

- ☐ One Git tag identifies code, configs, manifests, notebooks and deployment files.
- ☐ Colab and local environment reports are attached to runs; no hidden manual notebook edits are required.
- ☐ Large artifacts are outside Git and verified by immutable IDs/checksums.

Models

- ☐ Classification, detection and similarity each have a model card, evaluation report, preprocessing contract and golden set.
- ☐ PyTorch-to-ONNX parity passes; INT8 is enabled only where the per-task regression budget passes.
- ☐ Benchmark reports name hardware, runtime, precision, batch, warm-up and synchronization method.

Serving and pipelines

- ☐ FastAPI serves all synchronous contracts and reports the concrete model version/runtime.
- ☐ Celery batch jobs are idempotent, retry-bounded and observable; Redis keys are namespaced/TTL-bound.
- ☐ PostgreSQL migrations are repeatable; pgvector retrieval matches an exact-search oracle on fixtures.

Operations

- ☐ Readiness, metrics, structured logs, dashboards and alerts identify dependency/model failures.
- ☐ A/B assignment is stable and auditable; drift alerts do not auto-promote models.
- ☐ Promotion and rollback are tested using immutable bundles and image digests.

Security and review

- ☐ No credentials or sensitive data are stored in Git/notebooks/output; images run non-root where practical.
- ☐ Payload validation, auth/rate limits, dependency/image/secret scans and SBOM evidence exist.
- ☐ README contains a ten-minute reviewer path plus a deeper architecture/operations path.

Reviewer demo sequence

- Start the CPU Compose profile and show healthy services.
- Open MLflow and identify the champion versions/bundle evidence.
- Call classify, detect and similar endpoints; show model_version/runtime in responses.
- Submit a batch job; show Celery progress and PostgreSQL job record.
- Open Grafana; show latency/errors, queue and model-version panels.
- Promote a staged challenger in a controlled environment and demonstrate rollback.
- Explain where the Colab notebooks fit and why final TensorRT compilation belongs on the target GPU.

Risk register and troubleshooting guide

Risk	Impact	Control / fallback	Owner
Colab GPU unavailable/quota exhausted	Training/export cannot start	Run CPU preparation/tests locally; queue GPU work; use paid/dedicated GPU if deadline critical	ML engineer
Colab runtime disconnect	Progress/artifacts lost	Checkpoint to durable run folder; resume with hash validation; package early	ML engineer
Dependency/CUDA mismatch	Import/runtime errors	Pinned Colab requirements; environment report; clean runtime validation	ML/platform
Drive I/O/quota failure	Slow training or write errors	Archive/copy once to VM; avoid many small reads; atomic completed folder	ML engineer
ONNX parity failure	Different predictions	Test preprocessing, raw model output and postprocessing separately; inspect unsupported ops	ML engineer
INT8 quality regression	mAP/recall drops	Representative calibration; per-task gate; ship FP32 ONNX fallback	ML/QA
TensorRT engine incompatible	Engine fails to load on deployment GPU	Rebuild on target; record platform/GPU/TRT; retain ONNX source	Platform
Model bundle corruption	Unsafe/failed load	SHA-256, schema, size and trusted-source validation before deserialization	Backend/ platform
Redis outage/memory pressure	Batch/cache/rate limits fail	TTL/maxmemory, health alerts, retry/backpressure, PostgreSQL durable state	Backend
DB/pgvector migration mismatch	API startup/query failure	Alembic gate, backup/rollback, migration integration test	Backend
Model hot swap causes bad readiness	Traffic hits un-warmed model	Stage, verify, warm, atomic activation, rollback slot	Backend/ platform
Drift false positive	Wasteful or harmful retraining	Alert + investigation + labeled sample + approval; no autonomous promotion	ML/QA

Fast diagnostic order

Confirm Git SHA, config hash, dataset/split hash and bundle checksum.

Confirm device and exact runtime versions; print GPU name and memory only when CUDA is expected.

Run one golden sample through preprocessing → raw runtime → postprocessing, comparing each boundary.

Check readiness and structured logs before changing model code.

Reproduce with CPU ONNX when possible; isolate external services using tiny/fake runtime fixtures.

Preserve failed artifacts and environment reports; do not overwrite the evidence folder.

Sources and official guidance

Research checked 18 July 2026. Sources support product/runtime constraints; architecture and estimates are engineering recommendations for this challenge.

[S1] Applied Computing Technologies — ML Engineer Challenge repository
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/tree/main

[S2] Google Colab FAQ — runtime, GPU, storage, notebook format and resource limits
https://research.google.com/colaboratory/faq.html

[S3] Visual Studio Code — Jupyter Notebooks in VS Code
https://code.visualstudio.com/docs/datascience/jupyter-notebooks

[S4] PyTorch — Automatic Mixed Precision package (torch.amp)
https://docs.pytorch.org/docs/stable/amp.html

[S5] PyTorch — torch.onnx documentation
https://docs.pytorch.org/docs/stable/onnx.html

[S6] ONNX Runtime — Quantize ONNX models
https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html

[S7] MLflow — ML Model Registry and model aliases
https://mlflow.org/docs/latest/ml/model-registry/

[S8] NVIDIA TensorRT — Support Matrix, engine portability
https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/support-matrix.html

[S9] FastAPI — Lifespan events
https://fastapi.tiangolo.com/advanced/events/

[S10] Docker — Compose documentation
https://docs.docker.com/compose/

Versioning note

Package and platform releases change. Lock the tested versions in the repository, retain environment reports with each artifact, and rerun compatibility/parity tests before upgrading Colab dependencies, PyTorch, ONNX Runtime, TensorRT, MLflow or the serving image.